

A
Project Report
On

Fake Job Posting Detector

Submitted in partial fulfillment of the course
Supervised and Unsupervised Learning [24CAI0203]
of
Computer Science & Engineering (AI&ML)

during January-June, 2026

Submitted by:

2410993195 (Ishardeep Singh)
2410993183 (Damanjeet Singh)
2410993199 (Janvhi Dawra)
2410993202 (Jasmine Kaur)



CHITKARA UNIVERSITY

**CHANDIGARH-PATIALA NATIONAL HIGHWAY
RAJPURA (PATIALA) PUNJAB-140401 (INDIA)**

April & 2026

DECLARATION OF PROJECT

We, the undersigned students of the Department of Computer Science & Engineering – AI&ML at Chitkara University, Punjab, hereby declare that we have successfully completed the project titled **“Fake Job Posting Detector”** as part of the requirements for the Course **Supervised and Unsupervised Learning [24CAI0203]**.

We confirm that the project was conducted during the period 20 January, 2026 to 1 May, 2026 and has been prepared in partial fulfillment of the course *Supervised and Unsupervised Learning* for submission to the Department of Computer Science and Engineering (Artificial Intelligence).

We affirm that the work submitted is our original effort and has not been copied or reproduced from any other source, except where due acknowledgment has been made. We also confirm that all the resources, references, and materials used have been properly cited in the project.

We undertake to abide by the rules and regulations of Chitkara University, Punjab and accept full responsibility for the authenticity and validity of the project submitted.

Date: 29 April, 2026

Signed by:

2410993195, Ishardeep Singh
2410993183, Damanjeet Singh
2410993199, Janvhi Dawra
2410993202, Jasmine Kaur

.....
.....
.....
.....

Abstract

Fake job postings have become a significant threat on online recruitment platforms, leading to financial harm and misinformation among job seekers. This project addresses the problem by developing a machine learning-based system for detecting fraudulent job advertisements. A labeled dataset of real and fake job descriptions is preprocessed through text cleaning, stop-word removal, and TF-IDF vectorization to produce structured feature representations. Supervised classification models — Logistic Regression, Random Forest — are trained and evaluated using accuracy, precision, recall, F1-score, and confusion matrices. Comparative analysis across models demonstrates that text-based machine learning approaches can reliably distinguish fraudulent postings from legitimate ones. The findings suggest that such systems have practical potential for improving recruitment platform security and reducing exposure to online job scams.

Table of Contents

S.No	Contents	Page No.
1.	Introduction	5
2.	Problem Statement	6
3.	Methodology & Technical Details	7-8
4.	Dataset Description & Preparation	9-11
5.	Project Advantages & Limitation of Models	12-13
6.	Results & Discussions	14-19
7.	Conclusion with Future scope	20

Introduction

Online job portals and recruitment websites have become the primary medium for job searching. With the increase in online job applications, the number of fraudulent job advertisements has also increased. Fake job postings are created with the intention of stealing personal information, collecting money through scams, or misleading job seekers.

These fake advertisements appear very similar to legitimate job posts, making them difficult to identify manually. Many job seekers fall into these traps due to attractive salary offers, remote work promises, and urgent hiring messages.

This project aims to build an automated Fake Job Detection System using Machine Learning models. The system analyzes the textual content of job postings and classifies them as either **Real** or **Fake**. This project is motivated by the need for safe recruitment platforms and to protect users from job fraud.

Machine Learning provides an efficient solution to this problem by learning patterns from previously labeled job postings. Since job advertisements mainly contain textual information, Natural Language Processing (NLP) techniques are used to convert text into numerical features. In this project, TF-IDF vectorization is applied to represent job descriptions, and supervised learning models are trained to classify postings accurately. This automated approach helps in improving trust and security on online recruitment platforms.

Problem Statement

Online job platforms are increasingly exploited by fraudulent actors who post fake job listings to harvest personal information, solicit money, or deceive job seekers. This project builds a binary classification system to automatically distinguish real job postings from fraudulent ones using the **EMSI Fake Jobs dataset** containing **17,880 job postings**.

The core challenge is **extreme class imbalance** — only **4.57%** of postings are fraudulent — combined with the fact that the most informative signals are buried in unstructured text fields like job descriptions and company profiles, making this a mixed-modality classification problem requiring both NLP feature extraction and structured feature engineering.

Objectives:

The objectives of this project are:

1. To collect and analyze a dataset containing real and fake job postings.
2. To preprocess and clean job posting text data by handling missing values and removing unnecessary information.
3. To extract useful features from the dataset using TF-IDF vectorization and engineered numeric features.
4. To train supervised machine learning models such as Logistic Regression and Random Forest for classification.
5. To improve Logistic Regression performance using threshold tuning to achieve a better balance between precision and recall.
6. To evaluate the models using accuracy, precision, recall, F1-score, classification report, and confusion matrix.
7. To compare the performance of different models and identify the best-performing model for fraud detection.
8. To develop a reliable system that can predict whether a job posting is real or fraudulent.

Methodology & Technical Details

This project follows a standard machine learning pipeline for text classification and fraud detection.

Step 1: Data Loading

The dataset is loaded using the Pandas library into a DataFrame. The dataset contains job postings with multiple attributes such as job title, job description, requirements, company information, and a target label indicating whether the job post is real or fraudulent.

Step 2: Data Cleaning

Text data is cleaned and preprocessed to improve model performance. The cleaning process includes:

- Removing special characters and symbols
- Converting text to lowercase
- Removing stop words
- Removing unnecessary spaces

This ensures that irrelevant information does not affect the model training process.

Step 3: Feature Extraction

Since machine learning algorithms cannot directly process text data, the textual information is converted into numerical form using **TF-IDF Vectorization**.

TF-IDF (Term Frequency–Inverse Document Frequency) assigns weights to words based on their importance in a document relative to the entire dataset. Words that appear frequently in one job post but are rare in other job posts receive higher importance.

Step 4: Train-Test Splitting

The dataset is split into:

- **Training set (80%)**
- **Testing set (20%)**

This ensures that the model is evaluated on unseen data to test its generalization ability.

Step 5: Model Training

The following supervised machine learning models were trained in this project:

- **Logistic Regression (default threshold)**
- **Logistic Regression (threshold tuned)**
- **Random Forest (numeric features only)**

Logistic Regression is widely used in text classification problems because it performs well on high-dimensional sparse datasets like TF-IDF vectors. Random Forest was applied using numeric features only to compare its performance with the text-based approach.

Step 6: Threshold Tuning

By default, Logistic Regression classifies a job post as fake when the predicted probability is greater than **0.5**. In this project, **threshold tuning** was performed by changing the decision threshold to improve fraud detection performance, especially to reduce false negatives (fake jobs predicted as real). This method helps in achieving a better balance between precision and recall.

Step 7: Model Evaluation

The trained models were evaluated using the following performance metrics:

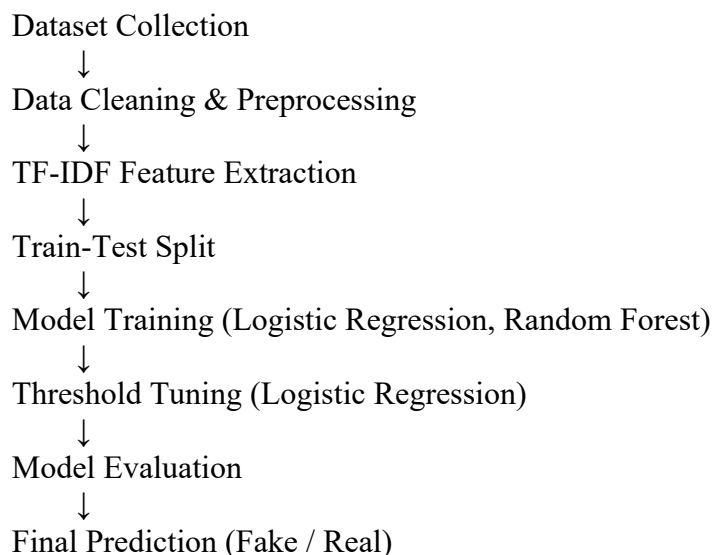
- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

These metrics help measure the model's effectiveness in correctly identifying fake job postings.

Step 8: Prediction

The final trained model can predict whether a new job posting is **Real** or **Fake** based on the learned patterns from the dataset.

SYSTEM ARCHITECTURE / FLOWCHART



Tools and Technologies Used

The project is implemented using **Python** in **Google Colab** environment. Libraries such as **Pandas** and **NumPy** are used for data processing, **Scikit-learn** is used for feature extraction and model training, and **Matplotlib/Seaborn** is used for visualizing results such as confusion matrices.

Dataset Description & Preparation

The dataset used in this project is `fake_job_postings.csv`, which contains job advertisements labeled as either real or fraudulent. The dataset was loaded directly from a GitHub repository using the Pandas library.

Each record in the dataset represents one job posting and contains multiple attributes such as job title, company profile, job description, requirements, benefits, employment type, industry, and other relevant job-related information. The dataset also includes a target column named `fraudulent`, which indicates whether a job post is fake or real.

The target variable values are:

- 0 → Real Job Posting
- 1 → Fake Job Posting

This dataset is suitable for text classification problems because it contains large textual fields such as description and requirements, which can be analyzed using Natural Language Processing (NLP) techniques.

Dataset Features

The dataset contains a combination of:

- Text features (`company_profile`, `description`, `requirements`, `benefits`)
- Categorical features (`location`, `department`, `employment_type`, `required_experience`, `required_education`, `industry`, `function`)
- Binary features (`telecommuting`, `has_company_logo`, `has_questions`)
- Target feature (`fraudulent`)

Handling Missing Values

The dataset contains missing values in multiple columns. To handle missing data, the notebook fills null values separately for text and categorical columns.

Text Columns

The following text columns were filled with empty strings (""):

- `company_profile`
- `description`
- `requirements`
- `benefits`

This ensures that text-processing techniques such as TF-IDF can be applied without errors.

Categorical Columns

The following categorical columns were filled with "Unknown":

- location
- department
- employment_type
- required_experience
- required_education
- industry
- function

This prevents missing values from affecting feature engineering and model training.

Feature Engineering (Numeric Feature Creation)

To improve model performance and capture additional patterns, several numeric features were created from the text fields.

The following features were engineered in the notebook:

- desc_length → number of characters in the job description
- req_length → number of characters in requirements
- company_profile_length → number of characters in company profile
- desc_word_count → number of words in description
- title_length → number of characters in job title
- title_word_count → number of words in job title
- company_profile_word_count → number of words in company profile
- requirements_word_count → number of words in requirements

These features help capture important information such as the length and detail level of job postings, which can be useful in detecting fake advertisements.

Text Feature Combination

To build a text-based classification model, the notebook combines multiple text columns into a single column. This combined column represents the full job posting content.

The text fields used were:

- company_profile
- description
- requirements
- benefits

This combined text data becomes the main input for TF-IDF vectorization.

Dataset Preparation for Machine Learning

After handling missing values and creating additional numeric features, the dataset was prepared for model training using the following steps:

1. Selecting the input features (text or numeric depending on model).
2. Selecting the target column (fraudulent).
3. Splitting the dataset into training and testing sets using an 80:20 ratio.
4. Applying TF-IDF vectorization for text-based classification models.
5. Using numeric engineered features for Random Forest model training.

TF-IDF Vectorization

For text classification, the notebook applies TF-IDF (Term Frequency–Inverse Document Frequency) vectorization to convert text data into numerical form. This technique gives higher weight to important words that appear frequently in a document but are rare across the dataset.

TF-IDF is highly effective for fraud detection because fake job postings often contain specific keywords such as “urgent hiring”, “registration”, “limited time”, etc.

Final Prepared Dataset

After preprocessing, the dataset is transformed into a suitable form for supervised machine learning algorithms. The cleaned and vectorized dataset is then used for training and evaluating models such as Logistic Regression and Random Forest.

Project Advantages & Limitation of Models

Advantages

1. Logistic Regression (Default Threshold)

- Logistic Regression performs very well for text classification problems, especially when TF-IDF features are used.
- It provides high recall, which is important for detecting fraudulent job postings.
- The model is fast to train and works efficiently on large datasets.
- It handles high-dimensional sparse text data effectively.

2. Logistic Regression (Threshold Tuned)

- Threshold tuning helps improve the fraud detection system by controlling the trade-off between precision and recall.
- It reduces the risk of missing fraudulent job postings (false negatives).
- It allows better customization of the model depending on the real-world requirement (for example, prioritizing recall over precision).
- It improves model reliability in sensitive applications like fraud detection.

3. Random Forest (Numeric Features Only)

- Random Forest is robust and performs well even when the dataset contains noise.
- It can capture complex non-linear relationships in numeric features.
- It reduces overfitting compared to a single decision tree.
- It provides good accuracy and stable performance.

Limitations

1. Logistic Regression (Default Threshold)

- Logistic Regression assumes linear separability, which may not always capture complex fraud patterns.
- Precision is relatively lower, meaning it may incorrectly classify some real job postings as fake.
- Default threshold (0.5) may not always be optimal for fraud detection.

2. Logistic Regression (Threshold Tuned)

- Choosing an incorrect threshold can negatively impact precision and lead to too many false alarms.
- Threshold tuning requires additional testing and analysis to select the best value.

- It may lead to bias toward detecting more fake jobs, sometimes misclassifying genuine postings.

3. Random Forest (Numeric Features Only)

- Since the model uses only numeric engineered features, it cannot fully capture important text-based fraud patterns.
- It achieved lower recall compared to Logistic Regression, meaning it misses more fraudulent job postings.
- Random Forest is computationally heavier compared to Logistic Regression.
- Interpretation of the model is more complex compared to linear models.

Results & Discussions

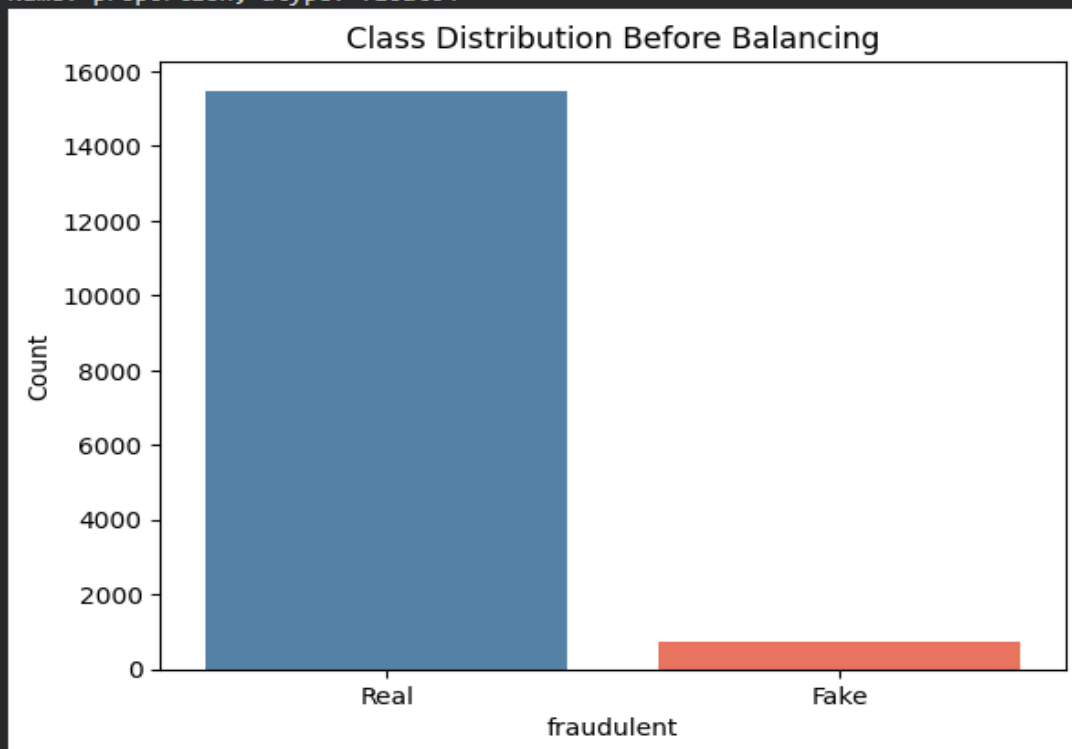
The performance of the trained machine learning models is evaluated for detecting fraudulent job postings. The models were tested using evaluation metrics such as accuracy, precision, recall, F1-score, classification report, and confusion matrix. Since the dataset is imbalanced, recall and F1-score are considered important for evaluating fraud detection performance.

6.1 Class Distribution

The dataset contains a large imbalance between real and fake job postings.

- **Real (0): 15481 (95.43%)**
- **Fake (1): 741 (4.57%)**

```
Class distribution (counts):  
fraudulent  
0    15481  
1     741  
Name: count, dtype: int64  
  
Class distribution (percentages):  
fraudulent  
0    95.43  
1     4.57  
Name: proportion, dtype: float64
```



This figure clearly shows that the dataset is highly imbalanced. The number of real job postings is much higher than fake postings. Because of this imbalance, accuracy alone is not sufficient to evaluate the model. Metrics such as recall and F1-score are necessary to understand how well the model detects fraudulent job postings.

6.2 Logistic Regression (Default Threshold) Results

The Logistic Regression model was trained using TF-IDF features and evaluated on the test dataset.

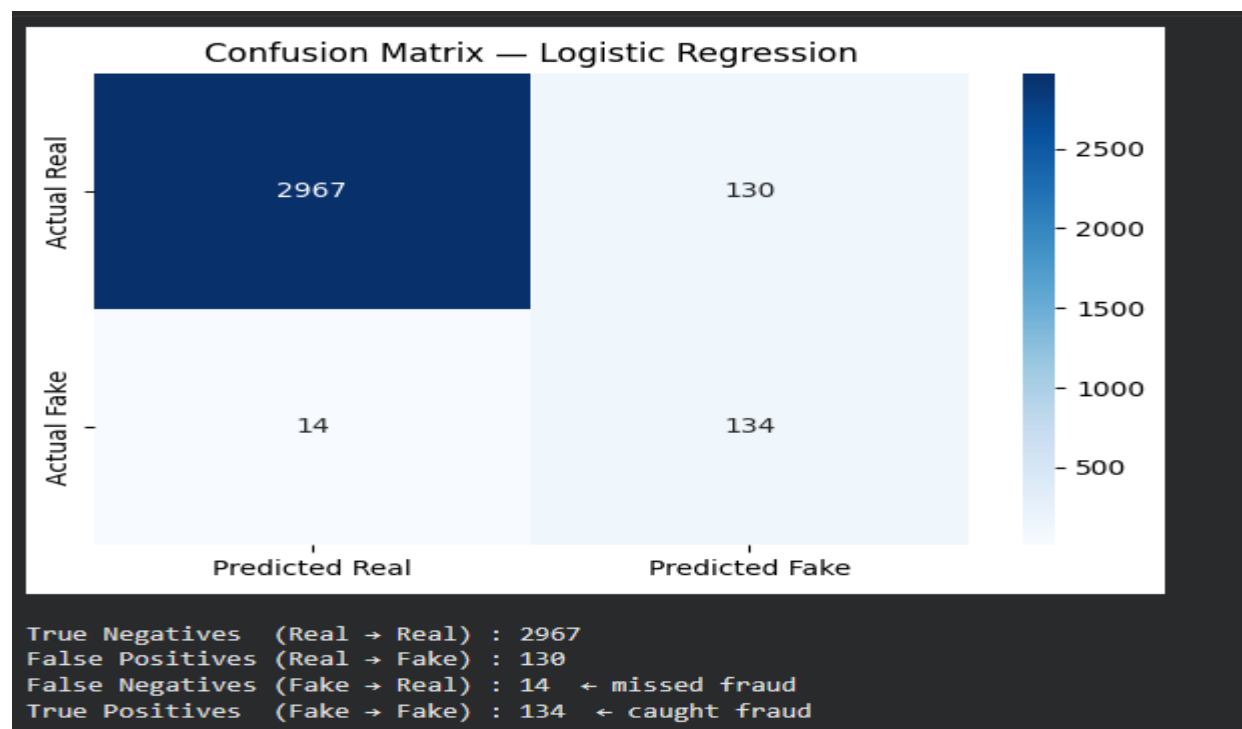
```
=====
LOGISTIC REGRESSION — EVALUATION
=====
Best C used      : 1.0
Best Penalty used : 12
Accuracy  : 0.9556
Precision : 0.5076
Recall    : 0.9054
F1 Score  : 0.6505

Detailed Report:

```

	precision	recall	f1-score	support
Real	1.00	0.96	0.98	3097
Fake	0.51	0.91	0.65	148
accuracy			0.96	3245
macro avg	0.75	0.93	0.81	3245
weighted avg	0.97	0.96	0.96	3245

The model achieved a very high recall of **90.54%**, which means it correctly detected most fake job postings. This is highly important in fraud detection systems, as missing a fraudulent posting can be harmful. However, the precision is moderate, which indicates that some real job postings were incorrectly classified as fake.

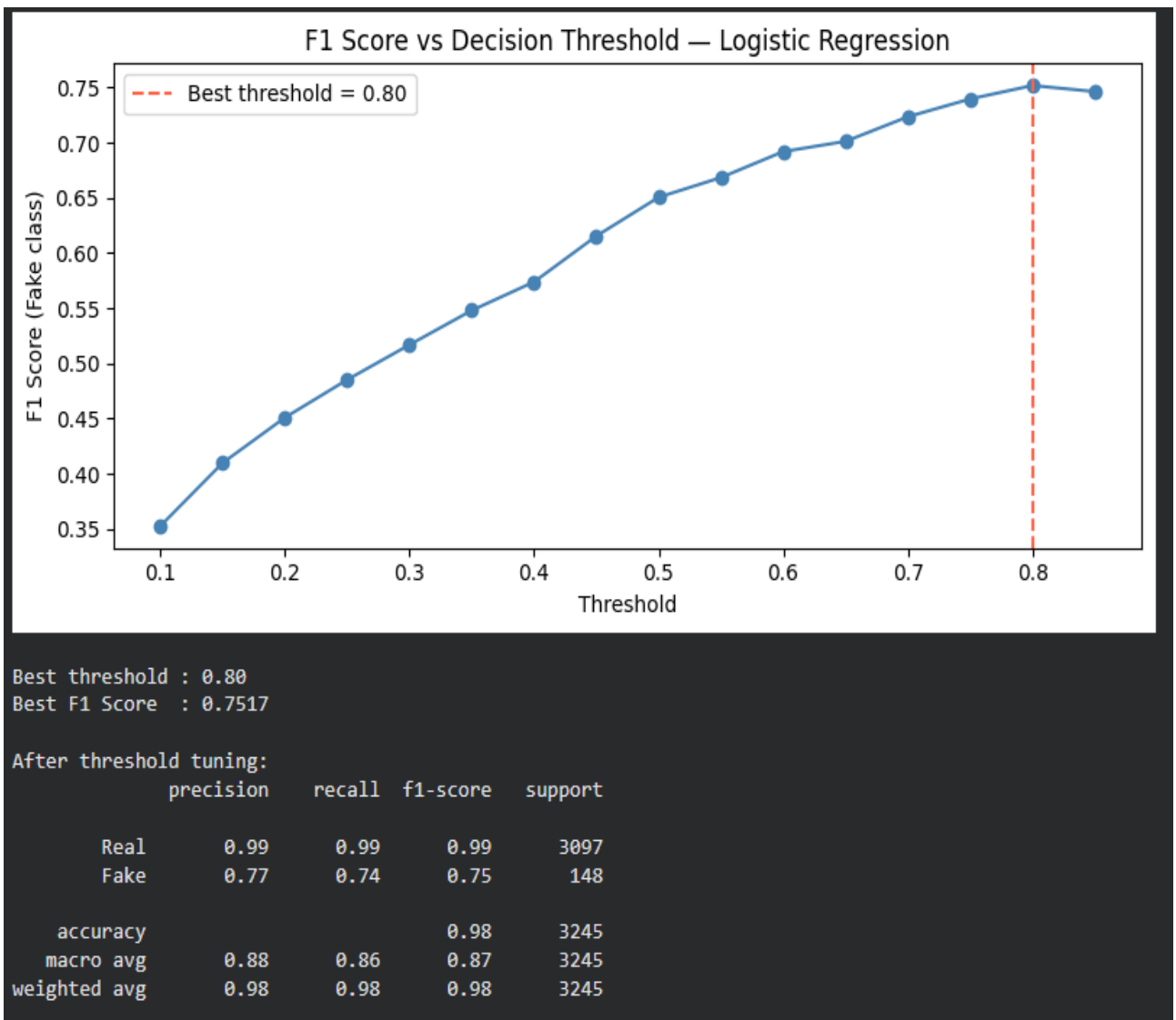


The confusion matrix shows that the model successfully catches most fraudulent postings (134 detected). Only 14 fraudulent postings were missed. This makes Logistic Regression highly suitable for fraud detection tasks where recall is prioritized.

6.3 Logistic Regression (Threshold Tuned) Results

To improve the overall model performance and reduce false alarms, threshold tuning was applied. Logistic Regression usually uses a default probability threshold of 0.5. In this project, the best threshold value was found to be:

- Best threshold = 0.80
- Best F1-score = 0.7517



Threshold tuning significantly improved precision from **50.76% to 76.76%**, meaning fewer real jobs were incorrectly classified as fake. Although recall decreased compared to the default model, the overall balance between precision and recall improved, resulting in a much higher F1-score.

This tuned model provides better overall fraud classification quality and reduces unnecessary false alerts.

6.4 Random Forest (Numeric Features Only) Results

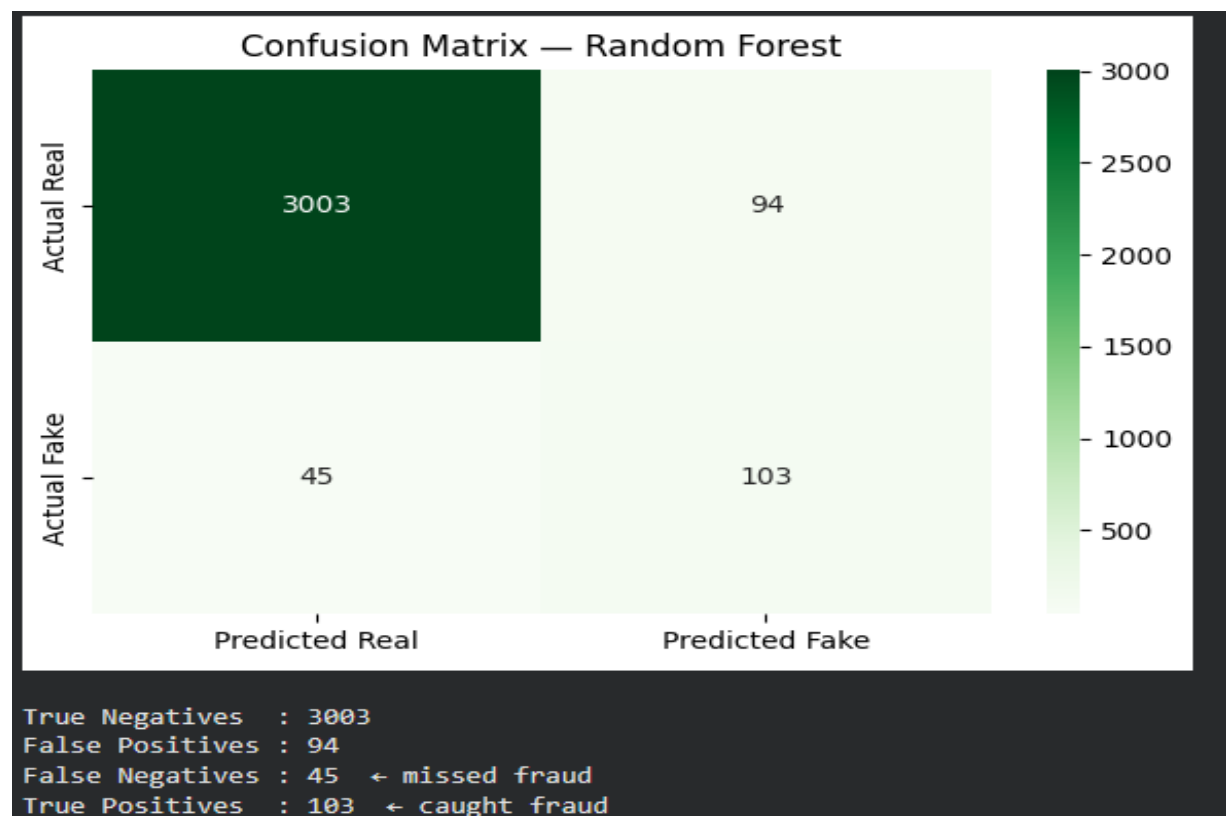
A Random Forest classifier was trained using only engineered numeric features such as text length and word count values.

```
=====
RANDOM FOREST — EVALUATION (Numeric Features)
=====
Accuracy  : 0.9572
Precision : 0.5228
Recall    : 0.6959
F1 Score  : 0.5971

Detailed Report:

```

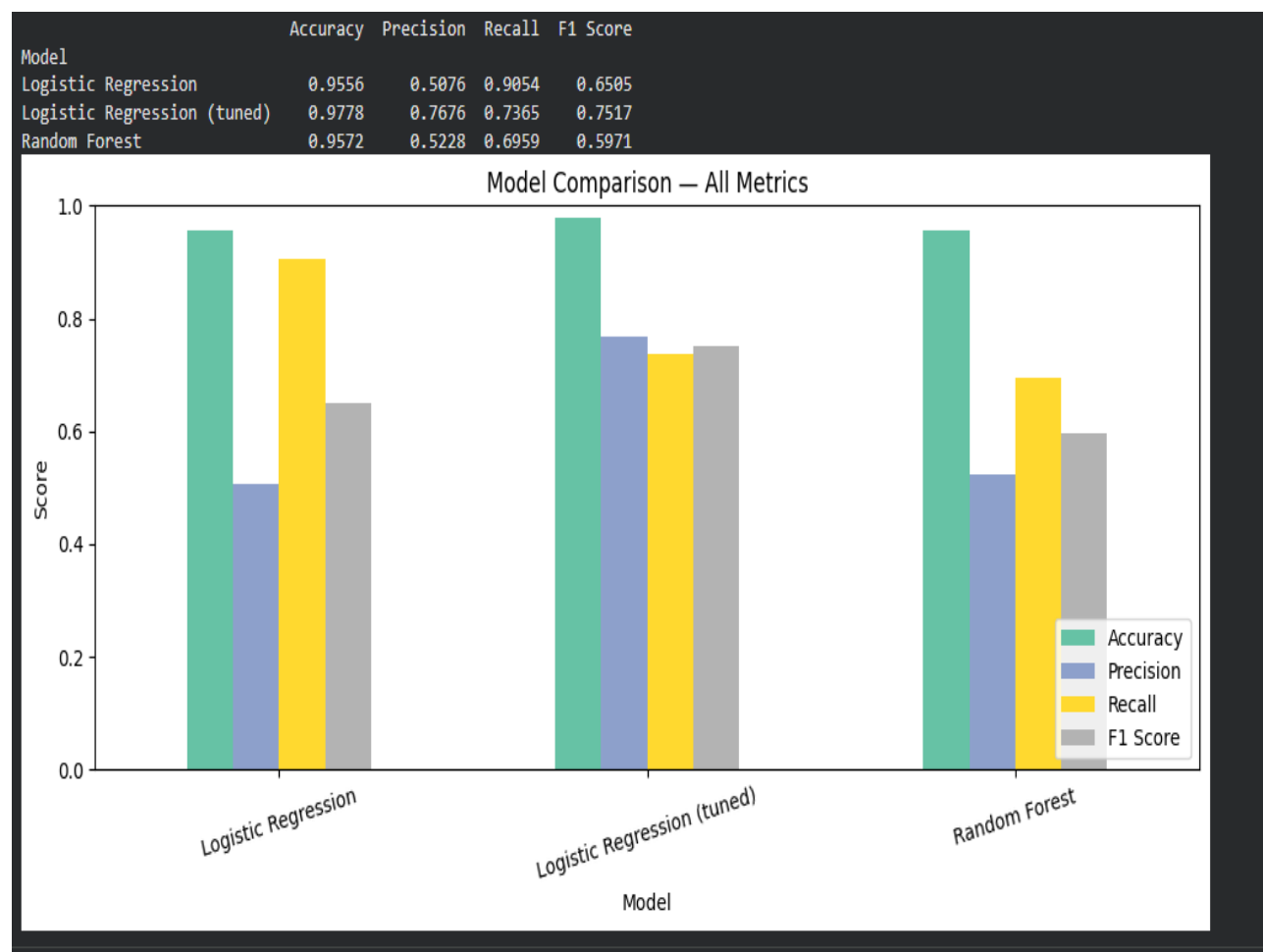
	precision	recall	f1-score	support
Real	0.99	0.97	0.98	3097
Fake	0.52	0.70	0.60	148
accuracy			0.96	3245
macro avg	0.75	0.83	0.79	3245
weighted avg	0.96	0.96	0.96	3245



The Random Forest model performs well in identifying real job postings but has a lower recall compared to Logistic Regression. It missed 45 fraudulent job postings, which is a limitation for fraud detection tasks. Since this model was trained only on numeric features, it cannot capture important fraud-related keywords and patterns in text data.

6.5 Model Comparison Table

The overall comparison of models is given below:



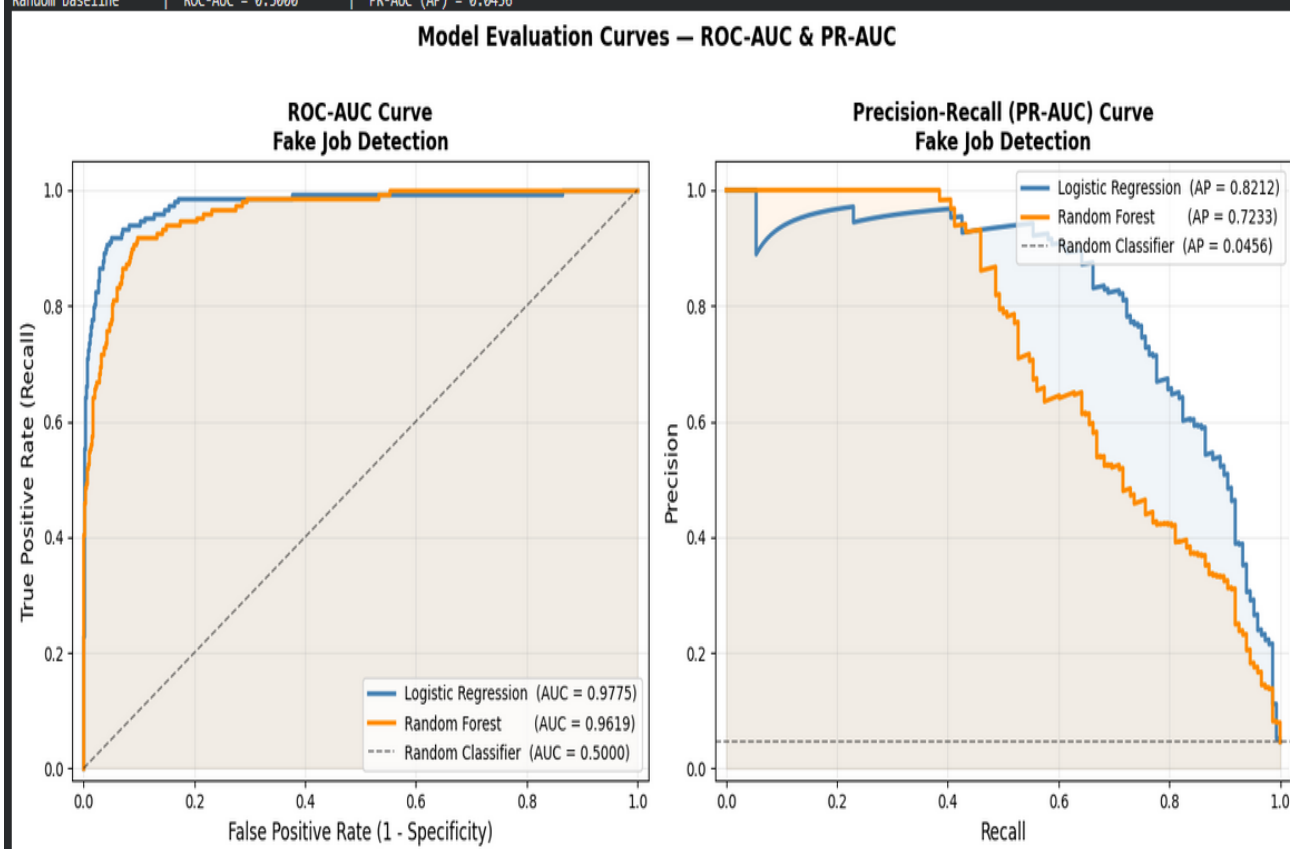
From the results, it is observed that:

- Logistic Regression (default threshold) gives the **highest recall (90.54%)**, meaning it catches maximum fraud cases.
- Logistic Regression (threshold tuned) gives the **best balanced performance**, with the highest F1-score (**75.17%**).
- Random Forest provides good accuracy but lower recall, making it less suitable compared to Logistic Regression for detecting fake job postings.

6.6 ROC-AUC and PR-AUC Analysis

To further evaluate the performance of the models on an imbalanced dataset, ROC-AUC and PR-AUC (Average Precision) scores were also calculated.

Logistic Regression	ROC-AUC = 0.9775	PR-AUC (AP) = 0.8212
Random Forest	ROC-AUC = 0.9619	PR-AUC (AP) = 0.7233
Random baseline	ROC-AUC = 0.5000	PR-AUC (AP) = 0.0456



ROC-AUC Interpretation:

LR AUC=0.9775 – correctly ranks a random Fake above a random Real 97.8% of the time
 RF AUC=0.9619 – lower because RF uses only 71 structured features vs 5,071 for LR

PR-AUC Interpretation:

Fraud rate (baseline AP) = 0.0456
 LR AP=0.8212 – 18.0x above random baseline on the fraud class
 RF AP=0.7233 – 15.9x above random baseline on the fraud class

PR-AUC is the stricter metric on this imbalanced dataset.

ROC-AUC can appear inflated when negatives heavily outnumber positives.

Both curves together confirm the model's learned discrimination is real.

The Logistic Regression model achieved ROC-AUC of **0.9775**, indicating strong discriminatory power between real and fake job postings. Random Forest achieved ROC-AUC of **0.9619**, which is slightly lower because it was trained only on structured numeric features.

Since the dataset is highly imbalanced, PR-AUC is a stricter metric. The baseline PR-AUC is equal to the fraud rate (**0.0456**). Logistic Regression achieved PR-AUC of **0.8212**, which is significantly higher than baseline, confirming strong fraud detection performance. Random Forest achieved PR-AUC of **0.7233**, which is also much higher than baseline.

6.6 Final Discussion

Fraud detection systems must prioritize identifying fraudulent cases correctly. Therefore, recall and F1-score are more important than accuracy. Logistic Regression performed the best overall, and threshold tuning further improved the system by increasing precision and F1-score. Hence, Logistic Regression with threshold tuning is considered the most reliable model for this project.

Conclusion with Future scope

Conclusion

This project demonstrates that fake job detection is an effective and solvable machine learning problem using classical supervised learning techniques. Logistic Regression trained on TF-IDF based text features along with engineered structured numeric features produced strong results in detecting fraudulent job postings. The dataset was properly processed and evaluated using appropriate validation methods.

Threshold tuning was performed to improve the model performance and achieve a better trade-off between precision and recall. The best decision threshold was found to be **0.80**, which achieved the best F1-score of **0.7517**. The tuned Logistic Regression model achieved an accuracy of **97.78%**, precision of **76.76%**, recall of **73.65%**, and F1-score of **75.17%**, making it the most reliable model in this project.

The results confirm that fraudulent job postings follow certain identifiable patterns such as incomplete company information, suspicious language, unrealistic offers, and misleading requirements. Therefore, the trained model can act as a strong first-level filter to reduce fraud and improve the security of online recruitment platforms.

Future Scope

The project can be further enhanced by implementing advanced techniques and adding more real-world fraud detection signals. Some possible future improvements are:

1. **Transformer-based Embeddings:**

Instead of TF-IDF, advanced NLP models such as BERT or similar transformer-based embeddings can be used. These embeddings capture semantic meaning, which can help detect fraud patterns even when different wording is used.

2. **Online Learning and Continuous Retraining:**

Fraudulent posting patterns change frequently. The model can be improved by retraining it incrementally with new job postings so that it remains updated against evolving scam techniques.

3. **Additional Structured Signals:**

More structured features can be added such as email domain verification and URL validation. Fake job postings often use suspicious or free domains which can serve as a strong fraud indicator.

4. **Explainable AI (Model Interpretability):**

Explainability tools such as SHAP values can be applied to explain why a specific job post was classified as fake. This will increase transparency and trust in the model.

5. **Advanced Hyperparameter Search:**

A complete hyperparameter search using techniques such as RandomizedSearchCV can be applied for better optimization of model parameters. With more computational resources, a wider range of parameters can be tested for improved accuracy and generalization.

References

1. Shivamb, 2017, “Real or Fake Job Posting Prediction Dataset”, Kaggle Dataset; <https://www.kaggle.com/datasets/shivamb/real-or-fake-fake-jobposting-prediction>
2. Géron, A., 2022, “Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow” (3rd Edition), O’Reilly Media.
3. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al., 2011, “Scikit-learn: Machine Learning in Python”, Journal of Machine Learning Research, 12, pp 2825–2830; <https://jmlr.org/papers/volume12/pedregosa11a/pedregosa11a.pdf>

Appendices

LOGISTIC REGRESSION

```
❏ lr_model = LogisticRegression(  
    C=best_C,  
    penalty=best_penalty,  
    max_iter=1000,  
    solver='saga',  
    random_state=42  
)  
  
lr_model.fit(X_train_resampled, y_train_resampled)  
  
y_pred_lr = lr_model.predict(X_test_combined)  
y_prob_lr = lr_model.predict_proba(X_test_combined)[:, 1]  
  
print("=" * 50)  
print("LOGISTIC REGRESSION - EVALUATION")  
print("=" * 50)  
print(f"Best C used      : {best_C}")  
print(f"Best Penalty used : {best_penalty}")  
print(f"Accuracy   : {accuracy_score(y_test, y_pred_lr):.4f}")  
print(f"Precision  : {precision_score(y_test, y_pred_lr):.4f}")  
print(f"Recall     : {recall_score(y_test, y_pred_lr):.4f}")  
print(f"F1 Score   : {f1_score(y_test, y_pred_lr):.4f}")  
print("\nDetailed Report:")  
print(classification_report(y_test, y_pred_lr, target_names=['Real', 'Fake']))
```

```
... =====  
LOGISTIC REGRESSION - EVALUATION  
=====
```

Best C used : 1.0
Best Penalty used : l2
Accuracy : 0.9556
Precision : 0.5076
Recall : 0.9054
F1 Score : 0.6505

Detailed Report:

	precision	recall	f1-score	support
Real	1.00	0.96	0.98	3097
Fake	0.51	0.91	0.65	148
accuracy			0.96	3245
macro avg	0.75	0.93	0.81	3245
weighted avg	0.97	0.96	0.96	3245

Threshold Tuning - Logistic Regression

```

thresholds = np.arange(0.1, 0.9, 0.05)
f1_scores_thresh = [f1_score(y_test, (y_prob_lr >= t).astype(int)) for t in thresholds]

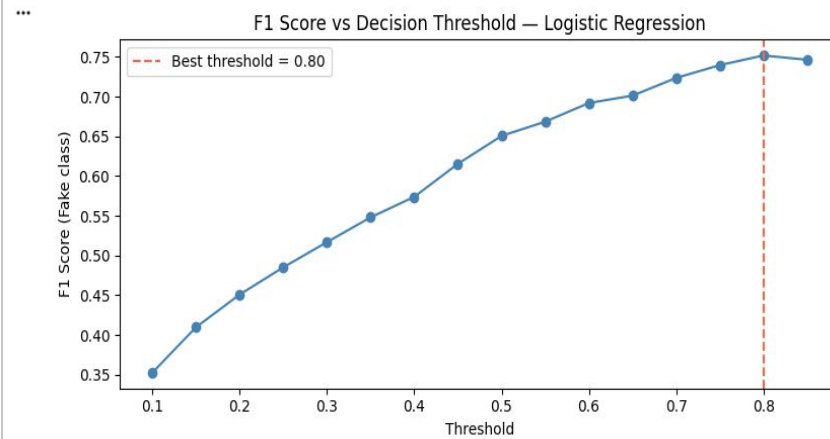
best_idx = np.argmax(f1_scores_thresh)
best_threshold = thresholds[best_idx]
best_f1 = f1_scores_thresh[best_idx]

plt.figure(figsize=(8, 4))
plt.plot(thresholds, f1_scores_thresh, marker='o', color='steelblue')
plt.axvline(x=best_threshold, color='tomato', linestyle='--', label=f'Best threshold = {best_threshold:.2f}')
plt.xlabel("Threshold")
plt.ylabel("F1 Score (Fake class)")
plt.title("F1 Score vs Decision Threshold - Logistic Regression")
plt.legend()
plt.tight_layout()
plt.show()

print(f"\nBest threshold : {best_threshold:.2f}")
print(f"Best F1 Score : {best_f1:.4f}")

y_pred_lr_tuned = (y_prob_lr >= best_threshold).astype(int)
print("\nAfter threshold tuning:")
print(classification_report(y_test, y_pred_lr_tuned, target_names=['Real', 'Fake']))

```



Best threshold : 0.80
Best F1 Score : 0.7517

After threshold tuning:

	precision	recall	f1-score	support
Real	0.99	0.99	0.99	3097
Fake	0.77	0.74	0.75	148
accuracy			0.98	3245
macro avg	0.88	0.86	0.87	3245
weighted avg	0.98	0.98	0.98	3245

RANDOM FOREST

```
rf_model = RandomForestClassifier(
    n_estimators=200,
    max_depth=20,
    min_samples_leaf=2,
    class_weight='balanced',
    random_state=42,
    n_jobs=-1
)

X_train_num_smote, y_train_num_smote = smote.fit_resample(
    csr_matrix(X_train_num_scaled), y_train
)

rf_model.fit(X_train_num_smote, y_train_num_smote)

y_pred_rf = rf_model.predict(X_test_num_scaled)
y_prob_rf = rf_model.predict_proba(X_test_num_scaled)[:, 1]

print("=" * 50)
print("RANDOM FOREST - EVALUATION (Numeric Features)")
print("=" * 50)
print(f"Accuracy : {accuracy_score(y_test, y_pred_rf):.4f}")
print(f"Precision : {precision_score(y_test, y_pred_rf):.4f}")
print(f"Recall : {recall_score(y_test, y_pred_rf):.4f}")
print(f"F1 Score : {f1_score(y_test, y_pred_rf):.4f}")
print("\nDetailed Report:")
print(classification_report(y_test, y_pred_rf, target_names=['Real', 'Fake']))
```

```
... =====
RANDOM FOREST - EVALUATION (Numeric Features)
=====
Accuracy : 0.9572
Precision : 0.5228
Recall : 0.6959
F1 Score : 0.5971

Detailed Report:

```

	precision	recall	f1-score	support
Real	0.99	0.97	0.98	3097
Fake	0.52	0.70	0.60	148
accuracy			0.96	3245
macro avg	0.75	0.83	0.79	3245
weighted avg	0.96	0.96	0.96	3245

```

from sklearn.metrics import roc_curve, roc_auc_score, precision_recall_curve, average_precision_score

# Computing curves
fpr_lr, tpr_lr, _ = roc_curve(y_test, y_prob_lr)
fpr_rf, tpr_rf, _ = roc_curve(y_test, y_prob_rf)

prec_lr_curve, rec_lr_curve, _ = precision_recall_curve(y_test, y_prob_lr)
prec_rf_curve, rec_rf_curve, _ = precision_recall_curve(y_test, y_prob_rf)

auc_lr = roc_auc_score(y_test, y_prob_lr)
auc_rf = roc_auc_score(y_test, y_prob_rf)
ap_lr = average_precision_score(y_test, y_prob_lr)
ap_rf = average_precision_score(y_test, y_prob_rf)
baseline_pr = y_test.mean() # random classifier PR baseline = fraud rate

print(f"Logistic Regression | ROC-AUC = {auc_lr:.4f} | PR-AUC (AP) = {ap_lr:.4f}")
print(f"Random Forest | ROC-AUC = {auc_rf:.4f} | PR-AUC (AP) = {ap_rf:.4f}")
print(f"Random baseline | ROC-AUC = 0.5000 | PR-AUC (AP) = {baseline_pr:.4f}")

fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# =====
# ROC Curve
# =====

ax = axes[0]
ax.plot(fpr_lr, tpr_lr, color='steelblue', lw=2.5,
        label=f'Logistic Regression (AUC = {auc_lr:.4f})')
ax.plot(fpr_rf, tpr_rf, color='darkorange', lw=2.5,
        label=f'Random Forest (AUC = {auc_rf:.4f})')
ax.plot([0,1],[0,1], 'k--', lw=1.2, alpha=0.5,
        label=f'Random Classifier (AUC = 0.5000)')
ax.fill_between(fpr_lr, tpr_lr, alpha=0.08, color='steelblue')
ax.fill_between(fpr_rf, tpr_rf, alpha=0.08, color='darkorange')
ax.set_xlabel('False Positive Rate (1 - Specificity)', fontsize=12)
ax.set_ylabel('True Positive Rate (Recall)', fontsize=12)
ax.set_title('ROC-AUC Curve\nFake Job Detection', fontsize=13, fontweight='bold')
ax.legend(fontsize=10, loc='lower right')
ax.set_xlim([-0.02, 1.02]); ax.set_ylim([-0.02, 1.05])
ax.grid(alpha=0.25)

# =====
# PR Curve
# =====

ax2 = axes[1]
ax2.plot(rec_lr_curve, prec_lr_curve, color='steelblue', lw=2.5,
        label=f'Logistic Regression (AP = {ap_lr:.4f})')
ax2.plot(rec_rf_curve, prec_rf_curve, color='darkorange', lw=2.5,
        label=f'Random Forest (AP = {ap_rf:.4f})')
ax2.axhline(y=baseline_pr, color='k', linestyle='--', lw=1.2, alpha=0.5,
        label=f'Random Classifier (AP = {baseline_pr:.4f})')
ax2.fill_between(rec_lr_curve, prec_lr_curve, alpha=0.08, color='steelblue')
ax2.fill_between(rec_rf_curve, prec_rf_curve, alpha=0.08, color='darkorange')
ax2.set_xlabel('Recall', fontsize=12)
ax2.set_ylabel('Precision', fontsize=12)
ax2.set_title('Precision-Recall (PR-AUC) Curve\nFake Job Detection', fontsize=13, fontweight='bold')
ax2.legend(fontsize=10, loc='upper right')
ax2.set_xlim([-0.02, 1.02]); ax2.set_ylim([-0.02, 1.05])
ax2.grid(alpha=0.25)

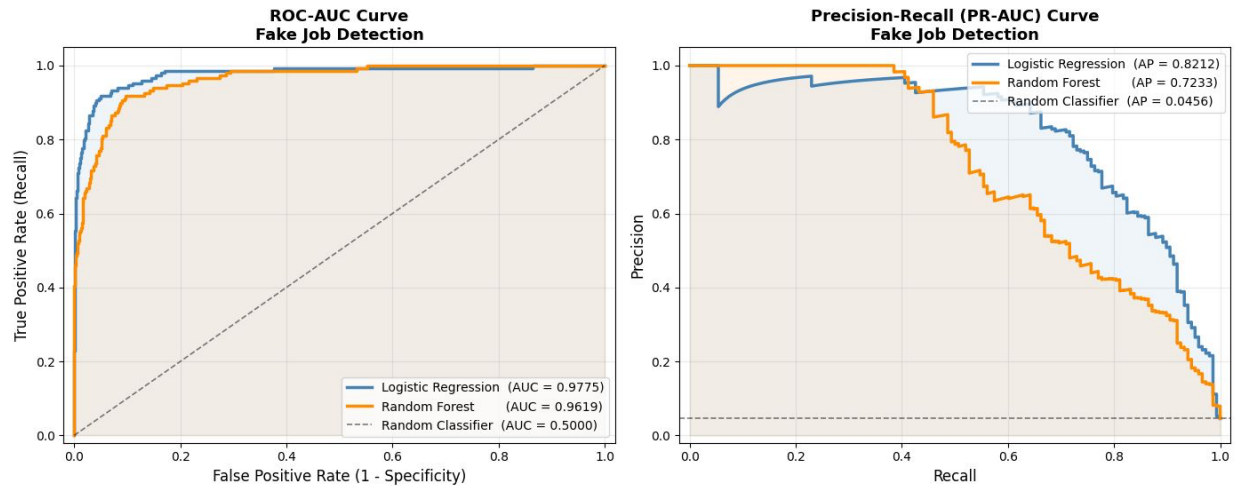
plt.suptitle('Model Evaluation Curves - ROC-AUC & PR-AUC', fontsize=14, fontweight='bold', y=1.01)
plt.tight_layout()
plt.show()

print("\nROC-AUC Interpretation:")
print(f" LR AUC={auc_lr:.4f} - correctly ranks a random Fake above a random Real {auc_lr*100:.1f}% of the time")
print(f" RF AUC={auc_rf:.4f} - lower because RF uses only 71 structured features vs 5,071 for LR")
print("\nPR-AUC Interpretation:")
print(f" Fraud rate (baseline AP) = {baseline_pr:.4f}")
print(f" LR AP={ap_lr:.4f} - {ap_lr/baseline_pr:.1f}x above random baseline on the fraud class")
print(f" RF AP={ap_rf:.4f} - {ap_rf/baseline_pr:.1f}x above random baseline on the fraud class")
print("\nPR-AUC is the stricter metric on this imbalanced dataset.")
print("ROC-AUC can appear inflated when negatives heavily outnumber positives.")
print("Both curves together confirm the model's learned discrimination is real.")

```

Logistic Regression	ROC-AUC = 0.9775	PR-AUC (AP) = 0.8212
Random Forest	ROC-AUC = 0.9619	PR-AUC (AP) = 0.7233
Random baseline	ROC-AUC = 0.5000	PR-AUC (AP) = 0.0456

Model Evaluation Curves — ROC-AUC & PR-AUC



ROC-AUC Interpretation:

LR AUC=0.9775 — correctly ranks a random Fake above a random Real 97.8% of the time
 RF AUC=0.9619 — lower because RF uses only 71 structured features vs 5,071 for LR

PR-AUC Interpretation:

Fraud rate (baseline AP) = 0.0456
 LR AP=0.8212 — 18.0x above random baseline on the fraud class
 RF AP=0.7233 — 15.9x above random baseline on the fraud class

PR-AUC is the stricter metric on this imbalanced dataset.
 ROC-AUC can appear inflated when negatives heavily outnumber positives.
 Both curves together confirm the model's learned discrimination is real.